

Министерство науки и высшего образования РФ  
Федеральное государственное автономное  
образовательное учреждение высшего образования  
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий  
институт

Программная инженерия  
кафедра

ОТЧЁТ О ПРАКТИЧЕСКОЙ РАБОТЕ №1

Конечные автоматы  
тема

Руководитель

подпись, дата

А.С. Кузнецов

инициалы, фамилия

Студент КИ24-16/16, 031939407

номер группы, зачётной книжки

подпись, дата

Ф.А. Жеравин

инициалы, фамилия

Красноярск 2026

## **1 Цель работы**

Реализация и исследование детерминированных и недетерминированных конечных автоматов.

## **2 Задачи**

1. Ознакомиться со сведениями по теории конечных автоматов (см. лекционный материал ДКА и НКА).

2. Получить у преподавателя собственный вариант задания, в котором должны быть указаны цепочки или наборы цепочек символов для распознавания ДКА и НКА.

3. Используя изученные механизмы, разработать в системе JFLAP согласно постановке задачи детерминированный конечный автомат, а также произвести программную реализацию на языке программирования Java (по согласованию с преподавателем Python, Си, C++). В случае невозможности создания ДКА, это должно доказываться формально. В коде программы обязательно наличие сущностей и процедур, относящихся к табличному представлению автомата. Использование функций обработки строковых данных запрещено. Результат работы, выдаваемый программой на экран, внешне должен быть схож, а фактически эквивалентен результату, выдаваемому JFLAP на тех же тестовых цепочках.

4. Используя изученные механизмы, разработать в системе JFLAP недетерминированный конечный автомат, а также произвести программную реализацию на языке программирования Java (по согласованию с преподавателем Python, Си, C++). В случае невозможности создания НКА, это должно доказываться формально. В коде программы обязательно наличие сущностей и процедур, относящихся к табличному представлению автомата. Использование функций обработки строковых данных запрещено. Результат работы, выдаваемый программой на экран, внешне должен быть схож, а фактически эквивалентен результату, выдаваемому JFLAP на тех же тестовых

цепочках.

5. Написать отчет и представить его путем загрузки в данный элемент курса на проверку преподавателю вместе с JFF-файлами и исходным кодом программ. Защита может проводиться в аудитории и дистанционно. Защита оценивается отдельно.

### **3 Ход работы**

Необходимо построить ДКА и НКА в системе JFLAP и произвести программную реализацию. В коде программы обязательно наличие сущностей и процедур, относящихся к табличному представлению автомата. Использование функций обработки строковых данных запрещено. Результат работы, выдаваемый программой на экран, внешне должен быть схож, а фактически эквивалентен результату, выдаваемому JFLAP на тех же тестовых цепочках.

#### **3.1 Задание**

Вариант №3.

##### **3.1.1 Постановка задачи а**

а) Построить ДКА, допускающий в алфавите  $\{0, 1\}$  все цепочки нулей и единиц, в которых символ 1 следует непосредственно за парой 00. Пример 1: Цепочки 100, 0010, 0010011001 принадлежат языку. Пример 2: цепочки 0001 и 0100 — не принадлежат.

##### **3.1.1.1 Замечание к постановке задачи**

В исходном задании формулировка условия неполная. После анализа примеров условие уточнено: строка не начинается с 01 и не содержит 000. При такой формулировке все примеры из задания сходятся.

##### **3.1.2 Постановка задачи б**

б) Построить НКА, допускающий цепочки в алфавите  $Z = \{0, 1\}$ , в которых содержится два нуля, разделенных позициями в количестве, кратном 4. Примечание: ноль тоже кратен 4.

### 3.1.3 Построение ДКА

Таблица 1 – Таблица переходов ДКА

|    | Переход по нулю | Переход по единице |
|----|-----------------|--------------------|
| q0 | q2              | q1                 |
| q1 | q4              | q1                 |
| q2 | q3              | <b>q6</b>          |
| q3 | <b>q6</b>       | q1                 |
| q4 | q5              | q1                 |
| q5 | <b>q6</b>       | q1                 |
| q6 | q6              | q6                 |

На рисунке 1 представлен граф переходов для ДКА, построенный с помощью JFLAP.

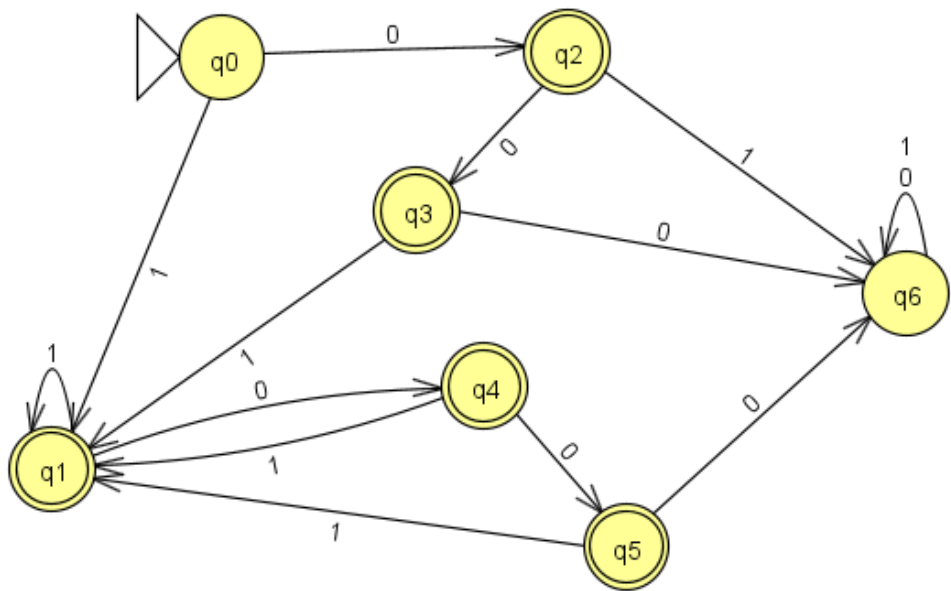


Рисунок 1 – Граф переходов ДКА

Перехват экрана при выполнении процесса распознавания нескольких тестовых цепочек представлен на рисунке 2.

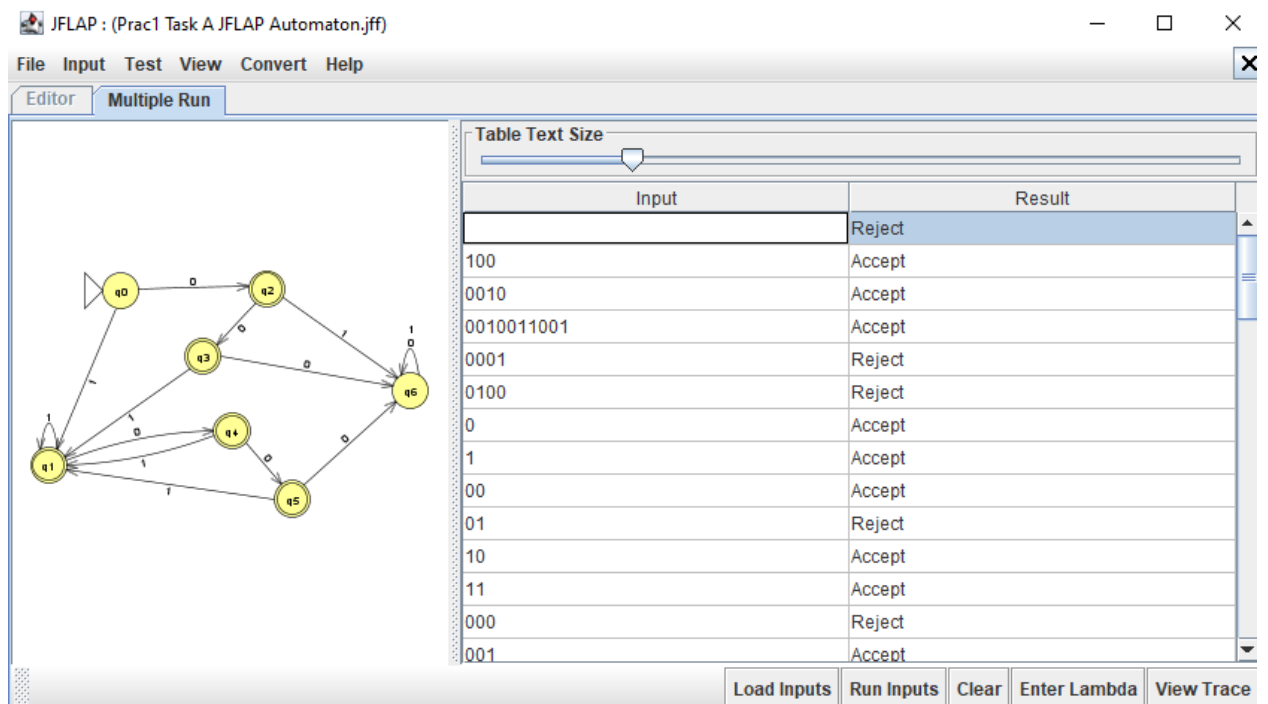


Рисунок 2 – Перехват экрана при распознавании нескольких цепочек ДКА

Строки со второй по шестую – строки из примера, автомат работает правильно с приведенными примерами.

### 3.1.3.1 Программная реализация ДКА

Код программы представлен в приложении А.

Результат работы программы представлен на рисунке 3.

```

H:\PPO\E-Commerce\Automata-Theory-Zheravin-FA\Prac 1>python "Prac1 Task A.py"
=====
Чтение цепочек из файла: test_strings_task_A.txt
=====

Цепочка: 100
  Путь: q0 -> q1 -> q4 -> q5
  Результат: ACCEPTED

Цепочка: 0010
  Путь: q0 -> q2 -> q3 -> q1 -> q4
  Результат: ACCEPTED

Цепочка: 0010011001
  Путь: q0 -> q2 -> q3 -> q1 -> q4 -> q5 -> q1 -> q1 -> q4 -> q5 -> q1
  Результат: ACCEPTED

Цепочка: 0001
  Путь: q0 -> q2 -> q3 -> q6 -> q6
  Результат: REJECTED

Цепочка: 0100
  Путь: q0 -> q2 -> q6 -> q6 -> q6
  Результат: REJECTED

Цепочка: 0
  Путь: q0 -> q2
  Результат: ACCEPTED

Цепочка: 1
  Путь: q0 -> q1
  Результат: ACCEPTED

Цепочка: 00
  Путь: q0 -> q2 -> q3
  Результат: ACCEPTED

Цепочка: 01
  Путь: q0 -> q2 -> q6
  Результат: REJECTED

Цепочка: 10
  Путь: q0 -> q1 -> q4
  Результат: ACCEPTED

Цепочка: 11
  Путь: q0 -> q1 -> q1
  Результат: ACCEPTED

Цепочка: 000
  Путь: q0 -> q2 -> q3 -> q6
  Результат: REJECTED

```

Рисунок 3 – Результат работы программы

### 3.1.4 Построение НКА

Таблица 2 – Таблица переходов НКА

|    | Переходы по 0 | Переходы по 1 |
|----|---------------|---------------|
| q0 | q0, q1        | q0            |
| q1 | q5, q2        | q2            |

Продолжение таблицы 2

|    | Переходы по 0 | Переходы по 1 |
|----|---------------|---------------|
| q2 | q3            | q3            |
| q3 | q4            | q4            |
| q4 | q1            | q1            |
| q5 | q5            | q5            |

На рисунке 4 представлен граф переходов для НКА, построенный с помощью JFLAP.

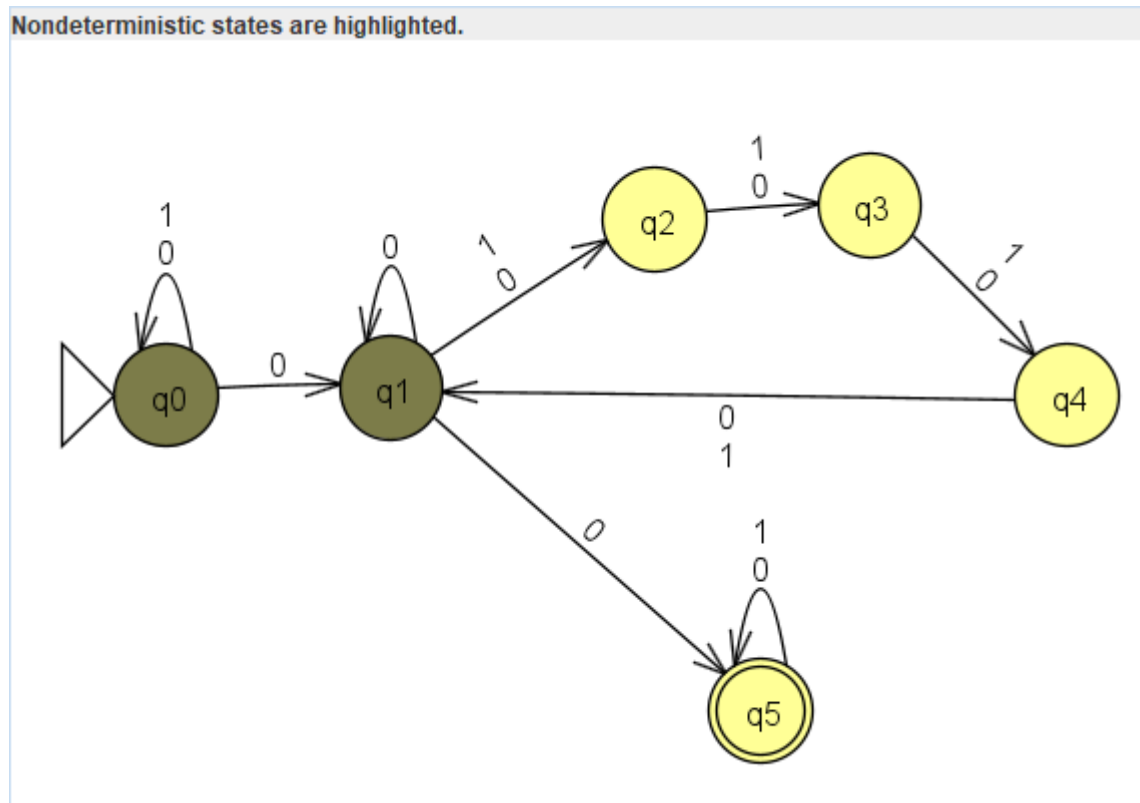


Рисунок 4 – Граф переходов НКА

Перехват экрана при выполнении процесса распознавания нескольких тестовых цепочек представлен на рисунке 5.

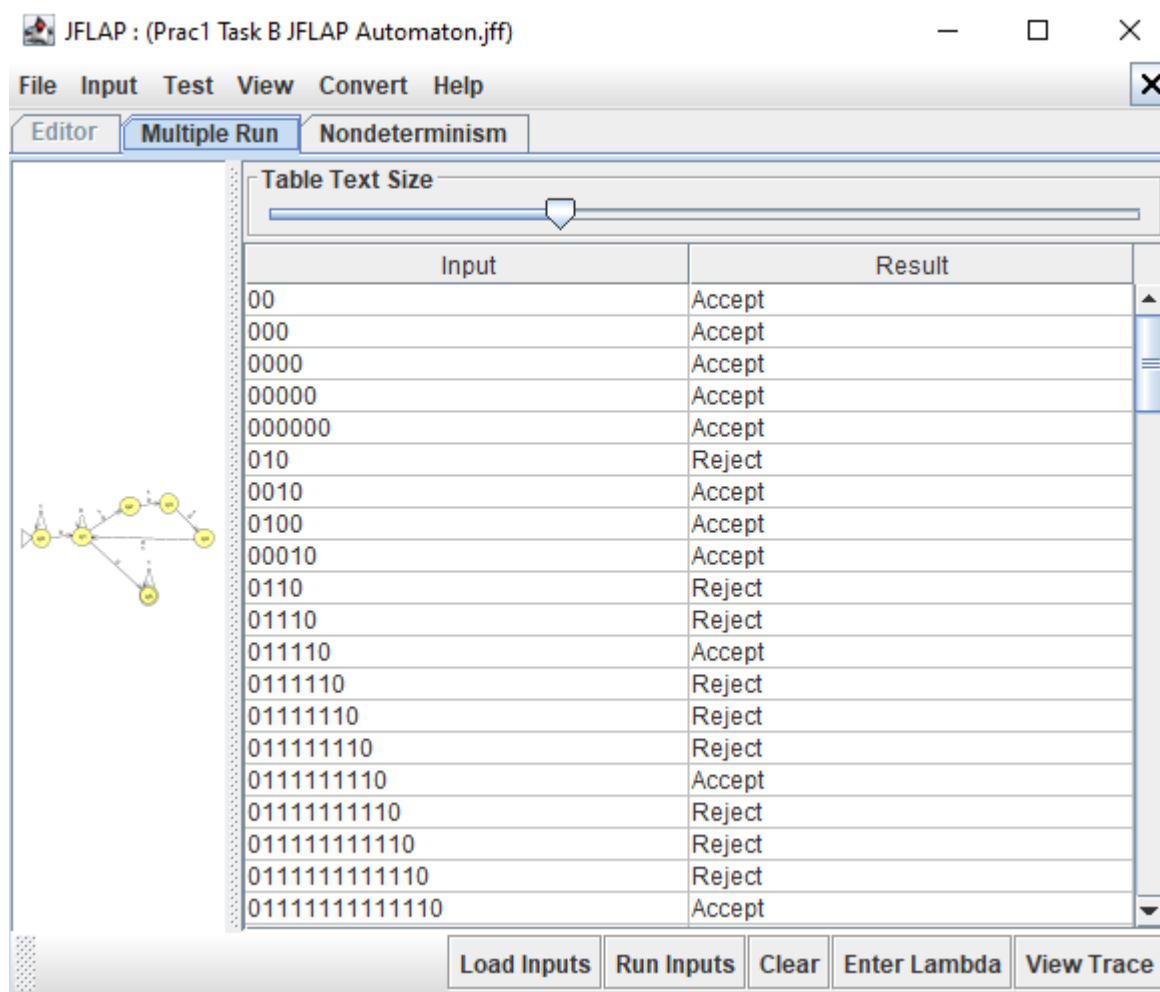


Рисунок 5 – Перехват экрана при распознавании нескольких цепочек НКА

### 3.1.4.1 Программная реализация НКА

Код программной реализации НКА представлен в приложении Б.

На рисунке 6 представлен частичный результат работы программы.

```

H:\PPO\E-Commerce\Automata-Theory-Zheravin-FA\Prac 1>python "Prac 1 Task B.py"
=====
Чтение цепочек из файла: test_strings_task_B.txt
=====

Цепочка: 00
Ошибка: недопустимый символ ' '
Результат: REJECTED

Цепочка: 000
Путь: q0 -> {q0,q1} -> {q0,q1,q2,q5} -> {q0,q1,q2,q3,q5}
Результат: ACCEPTED

Цепочка: 0000
Путь: q0 -> {q0,q1} -> {q0,q1,q2,q5} -> {q0,q1,q2,q3,q5} -> {q0,q1,q2,q3,q4,q5}
Результат: ACCEPTED

Цепочка: 00000
Путь: q0 -> {q0,q1} -> {q0,q1,q2,q5} -> {q0,q1,q2,q3,q5} -> {q0,q1,q2,q3,q4,q5} -> {q0,q1,q2,q3,q4,q5}
Результат: ACCEPTED

Цепочка: 000000
Путь: q0 -> {q0,q1} -> {q0,q1,q2,q5} -> {q0,q1,q2,q3,q5} -> {q0,q1,q2,q3,q4,q5} -> {q0,q1,q2,q3,q4,q5}
Результат: ACCEPTED

Цепочка: 010
Путь: q0 -> {q0,q1} -> {q0,q2} -> {q0,q1,q3}
Результат: REJECTED

Цепочка: 0010
Путь: q0 -> {q0,q1} -> {q0,q1,q2,q5} -> {q0,q2,q3,q5} -> {q0,q1,q3,q4,q5}
Результат: ACCEPTED

Цепочка: 0100
Путь: q0 -> {q0,q1} -> {q0,q2} -> {q0,q1,q3} -> {q0,q1,q2,q4,q5}
Результат: ACCEPTED

Цепочка: 00010
Путь: q0 -> {q0,q1} -> {q0,q1,q2,q5} -> {q0,q1,q2,q3,q5} -> {q0,q2,q3,q4,q5} -> {q0,q1,q3,q4,q5}
Результат: ACCEPTED

Цепочка: 0110
Путь: q0 -> {q0,q1} -> {q0,q2} -> {q0,q3} -> {q0,q1,q4}
Результат: REJECTED

```

Рисунок 6 – Результат работы программы для НКА

## 4 Вывод

В ходе практической работы были изучены теория конечных автоматов и механизмы построения детерминированных и недетерминированных конечных автоматов в системе JFLAP.

## ПРИЛОЖЕНИЕ А

### Код программной реализации ДКА

```
import os

TRANSITIONS = [
    {0: 2, 1: 1}, # q0 – начало строки
    {0: 4, 1: 1}, # q1 – первый символ "1"
    {0: 3, 1: 6}, # q2 – первый символ "0"
    {0: 6, 1: 1}, # q3 – начало "00"
    {0: 5, 1: 1}, # q4 – один "0" в конце
    {0: 6, 1: 1}, # q5 – два "0" в конце
    {0: 6, 1: 6}, # q6 – ловушка
]

INITIAL_STATE = 0
FINAL_STATES = {1, 2, 3, 4, 5}
STATE_NAMES = ["q0", "q1", "q2", "q3", "q4", "q5", "q6"]

INPUT_FILE = "test_strings_task_A.txt"

def recognize(chain):
    """
    Прогоняет цепочку по ДКА.
    Возвращает (accepted: bool, path: list[str], error: str|None).
    Без строковых функций – только посимвольный обход.
    """
    state = INITIAL_STATE
    path = [STATE_NAMES[state]]

    for ch in chain:
        if ch == '0':
            symbol = 0
        elif ch == '1':
            symbol = 1
        else:
            return False, path, f"недопустимый символ '{ch}'"

        state = TRANSITIONS[state][symbol]
        path.append(STATE_NAMES[state])

    accepted = state in FINAL_STATES
    return accepted, path, None

def process_and_print(chain):
    """Прогоняет цепочку и печатает результат в формате, похожем на JFLAP."""
    accepted, path, error = recognize(chain)
```

```

display = chain if chain != "" else "(пустая)"
print(f"\nЦепочка: {display}")
if error is not None:
    print(f" Ошибка: {error}")
    print(" Результат: REJECTED")
else:
    print(f" Путь: {' -> '.join(path)}")
    print(f" Результат: {'ACCEPTED' if accepted else 'REJECTED'}")

def run_from_file(filename):
    """Считывает цепочки из файла (по одной на строку) и прогоняет каждую."""
    if not os.path.exists(filename):
        print(f"Файл '{filename}' не найден. Переход в интерактивный режим.")
        return False

    print("=" * 60)
    print(f"Чтение цепочек из файла: {filename}")
    print("=" * 60)

    with open(filename, "r", encoding="utf-8") as f:
        for line in f:
            chain = ""
            for ch in line:
                if ch != '\n' and ch != '\r':
                    chain += ch
            if chain == "":
                continue
            process_and_print(chain)

    return True

def run_interactive():
    """Интерактивный режим: ввод цепочек с консоли."""
    print("=" * 60)
    print("ДКА: нет '000' и нет начала '01'")
    print("Алфавит: {0, 1}")
    print("Введите цепочку или 'exit' для выхода.")
    print("=" * 60)

    while True:
        try:
            chain = input("\n> ").strip()
        except EOFError:
            break

        if chain == "exit":
            print("Выход.")
            break

```

```
        process_and_print(chain)

def main():
    if not run_from_file(INPUT_FILE):
        run_interactive()
    else:
        print("\n" + "=" * 60)
        print("Файл обработан. Переход в интерактивный режим.")
        print("Введите цепочку или 'exit' для выхода.")
        print("=" * 60)
        run_interactive()

if __name__ == "__main__":
    main()
```

## ПРИЛОЖЕНИЕ Б

### Код программной реализации НКА

```
import os

TRANSITIONS = {
    (0, 0): [0, 1], (0, 1): [0],
    (1, 0): [5, 2], (1, 1): [2],
    (2, 0): [3], (2, 1): [3],
    (3, 0): [4], (3, 1): [4],
    (4, 0): [1], (4, 1): [1],
    (5, 0): [5], (5, 1): [5],
}

INITIAL_STATE = 0
FINAL_STATES = {5}
STATE_NAMES = ["q0", "q1", "q2", "q3", "q4", "q5"]

INPUT_FILE = "test_strings_task_B.txt"

def recognize(chain):
    """
    Прогоняет цепочку по НКА.
    Возвращает (accepted: bool, path: list[str], error: str|None).
    Имитация всех копий НКА через множество текущих состояний.
    """
    current = {INITIAL_STATE}
    path = [STATE_NAMES[INITIAL_STATE]]

    for ch in chain:
        if ch == '0':
            symbol = 0
        elif ch == '1':
            symbol = 1
        else:
            return False, path, f"недопустимый символ '{ch}'"

        next_states = set()
        for state in current:
            if (state, symbol) in TRANSITIONS:
                for ns in TRANSITIONS[(state, symbol)]:
                    next_states.add(ns)

        if not next_states:
            return False, path, None

        current = next_states
        path.append("{ " + ", ".join(STATE_NAMES[s]
```

```

        for s in sorted(current)) + "}")

    accepted = len(current & FINAL_STATES) > 0
    return accepted, path, None

def process_and_print(chain):
    """Прогоняет цепочку и печатает результат."""
    accepted, path, error = recognize(chain)
    display = chain if chain != "" else "(пустая)"
    print(f"\nЦепочка: {display}")
    if error is not None:
        print(f" Ошибка: {error}")
        print(" Результат: REJECTED")
    else:
        print(f" Путь: {' -> '.join(path)}")
        print(f" Результат: {'ACCEPTED' if accepted else 'REJECTED'}")

def run_from_file(filename):
    """Считывает цепочки из файла (по одной на строку) и прогоняет каждую."""
    if not os.path.exists(filename):
        print(f"Файл '{filename}' не найден. Переход в интерактивный режим.")
        return False

    print("=" * 60)
    print(f"Чтение цепочек из файла: {filename}")
    print("=" * 60)

    with open(filename, "r", encoding="utf-8") as f:
        for line in f:
            chain = ""
            for ch in line:
                if ch != '\n' and ch != '\r':
                    chain += ch
            if chain == "":
                continue
            process_and_print(chain)

    return True

def run_interactive():
    """Интерактивный режим: ввод цепочек с консоли."""
    print("=" * 60)
    print("НКА: два нуля, между которыми число символов кратно 4")
    print("Алфавит: {0, 1}")
    print("Введите цепочку или 'exit' для выхода.")
    print("=" * 60)

    while True:

```

```
        try:
            chain = input("\n> ").strip()
        except EOFError:
            break

        if chain == "exit":
            print("Выход.")
            break

        process_and_print(chain)

def main():
    if not run_from_file(INPUT_FILE):
        run_interactive()
    else:
        print("\n" + "=" * 60)
        print("Файл обработан. Переход в интерактивный режим.")
        print("Введите цепочку или 'exit' для выхода.")
        print("=" * 60)
        run_interactive()

if __name__ == "__main__":
    main()
```